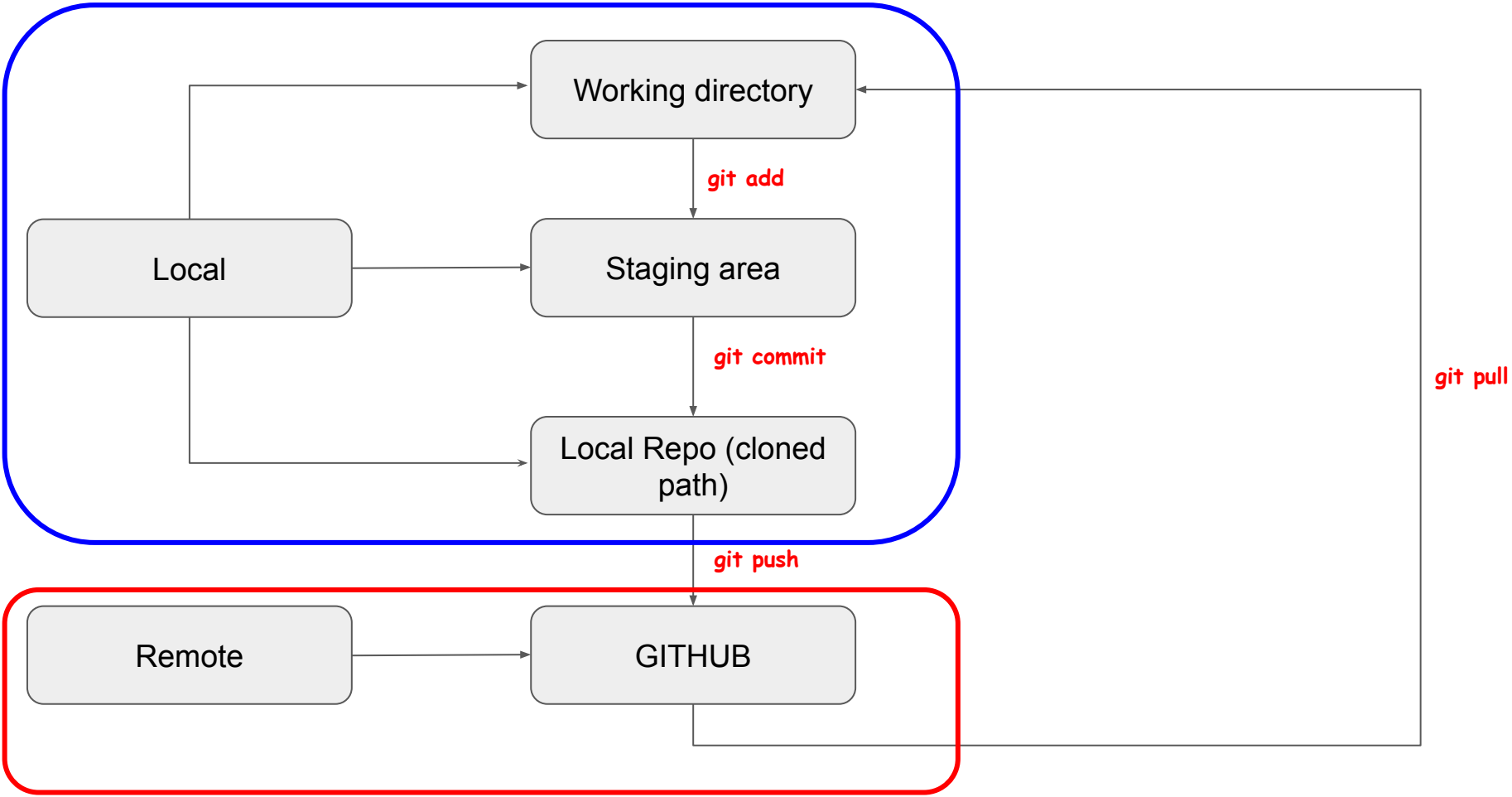
GIT

Watch this recording before proceeding further


Working directory
git add
Local
Staging area
git commit
Local Repo (cloned path)
git push
Remote
GITHUB
git pull

How to bypass a common git push problem?

```
Username for 'https://github.com': a
Password for 'https://a@github.com':
```

GIT approaches

1. **Store the Credentials applicable for PAT:** Use Git's credential helper to store your credentials in a file.
   `git config --global credential.helper store`

- After passing the above command, for the first push git asks for username and password.
- Here we need to pass our username and PAT that was generated.
- For the rest of the pushes it won't ask as a new file .git-credentials will be created and it stores your username and PAT

2. **SSH Update remote URL:**
   `git remote set-url origin git@github.com:username/repo.git`

<TASK>

`git config --global credential.helper store`

Immediate git push

Stores the information in .git-credentials

No need of username and pwd for future pushes

Some basic GIT commands

- Git diff

* Track the changes that have not been staged:
$ git diff

* Track the changes that have staged but not committed:
$ git diff --staged
$ git diff --cached

Some basic GIT commands

- Commit history

1. Git log

   Display the most recent commits and the status of the head:
   $ git log

   Display the output as one commit per line:
   $ git log –oneline

   Displays the files that have been modified:
   $ git log –stat

   Display the modified files with location:
   $ git log -p

Some basic GIT commands

- .gitignore

* Specify intentionally untracked files that Git should ignore.

Create .gitignore:
$ touch .gitignore

* List of ignored files: $ git ls-files -i --exclude-standard

GIT Branching

Git branch

Create branch:
$ git branch <branch name>

List Branch:
$ git branch --list

Delete Branch:
$ git branch -d<branch name>

Delete a remote Branch:
$ git push origin -delete <branch name>

GIT Branching

Rename Branch:
$ git branch -m <old branch name><new branch name>

Git checkout
Switch between branches in a repository.

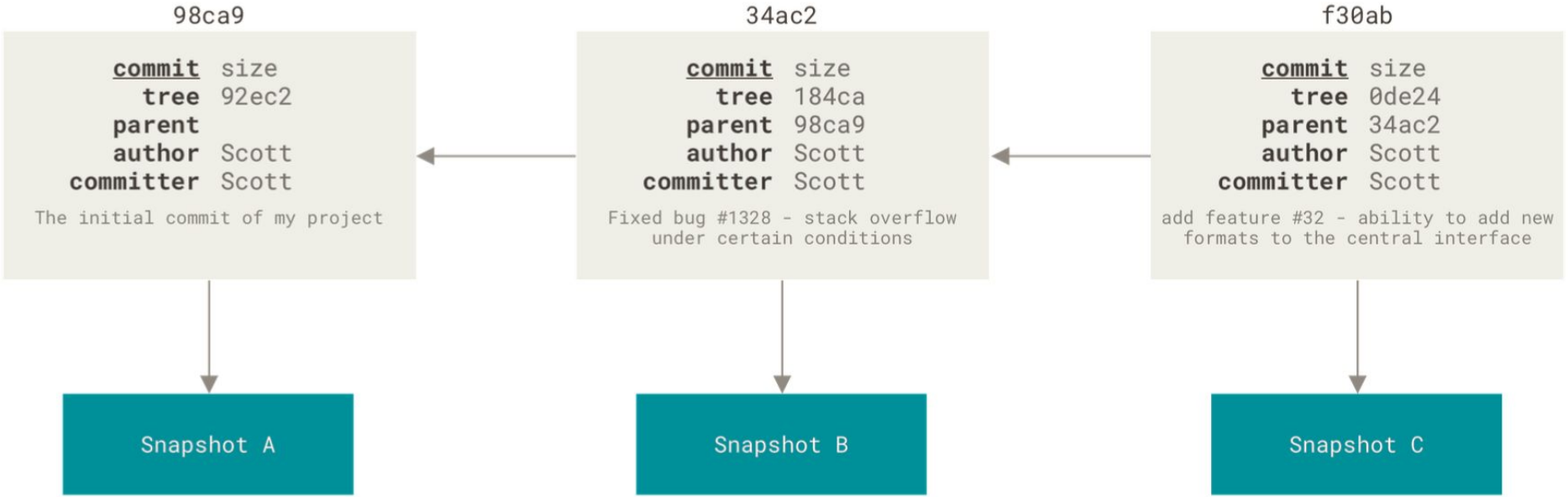
Switch to a particular branch:
$ git checkout <branch name>

Create a new branch and switch to it:
$ git checkout -b <branchname>

Checkout a Remote branch:
$ git checkout <remotebranch>

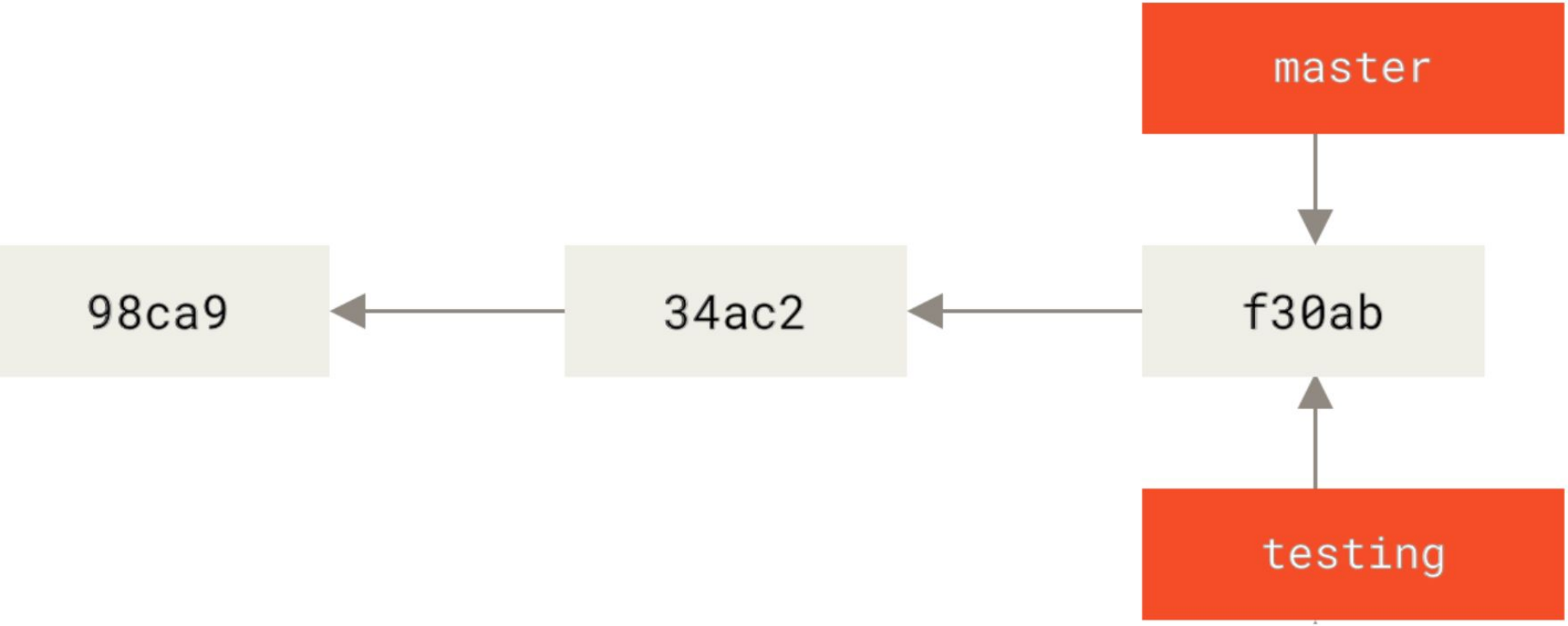
Branches in a Nutshell

If you make some changes and commit again, the next commit stores a pointer to the commit that came immediately before it.
Git doesn’t store data as a series of changesets or differences, but instead as a series of snapshots.

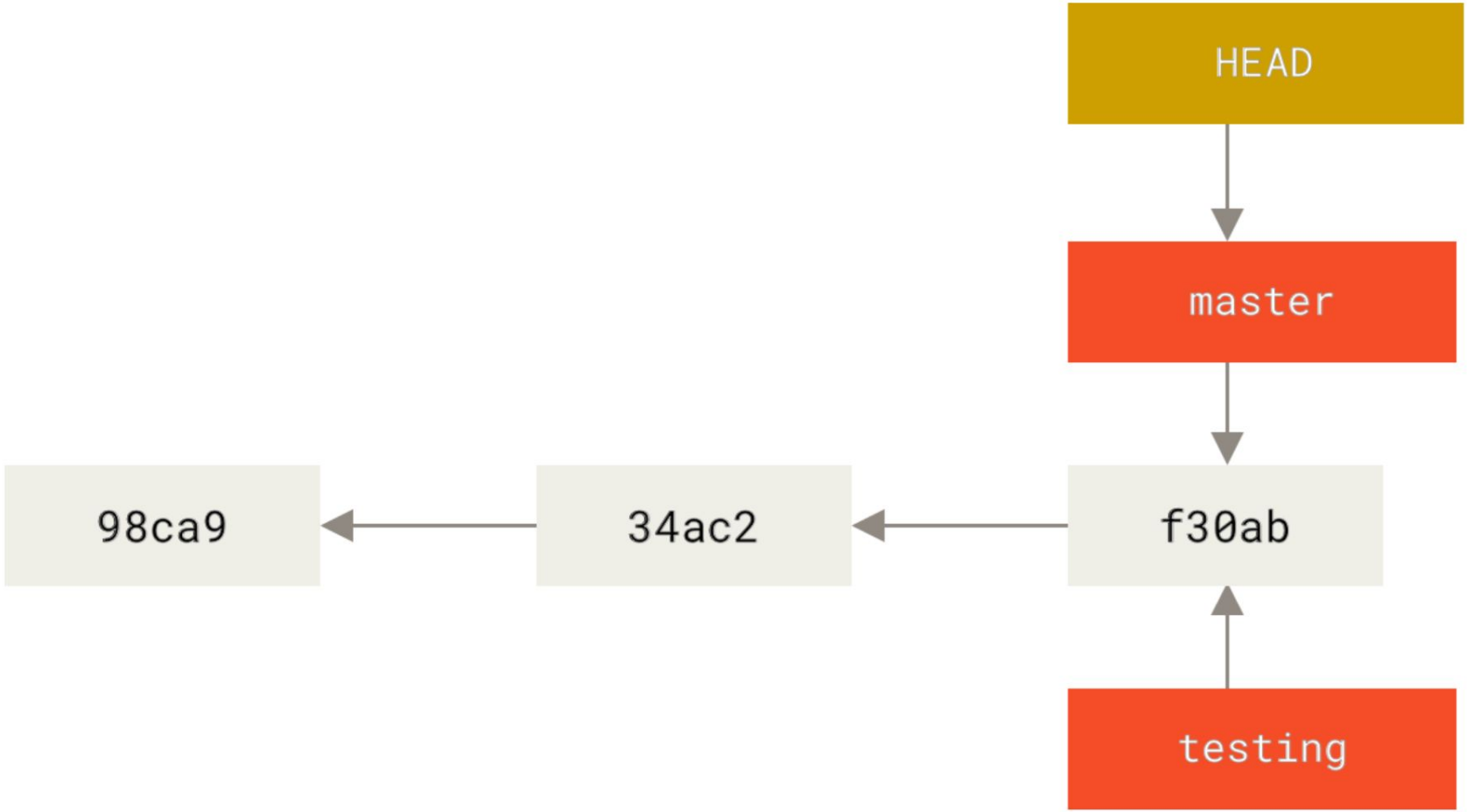


Creating a New Branch

What happens when you create a new branch?

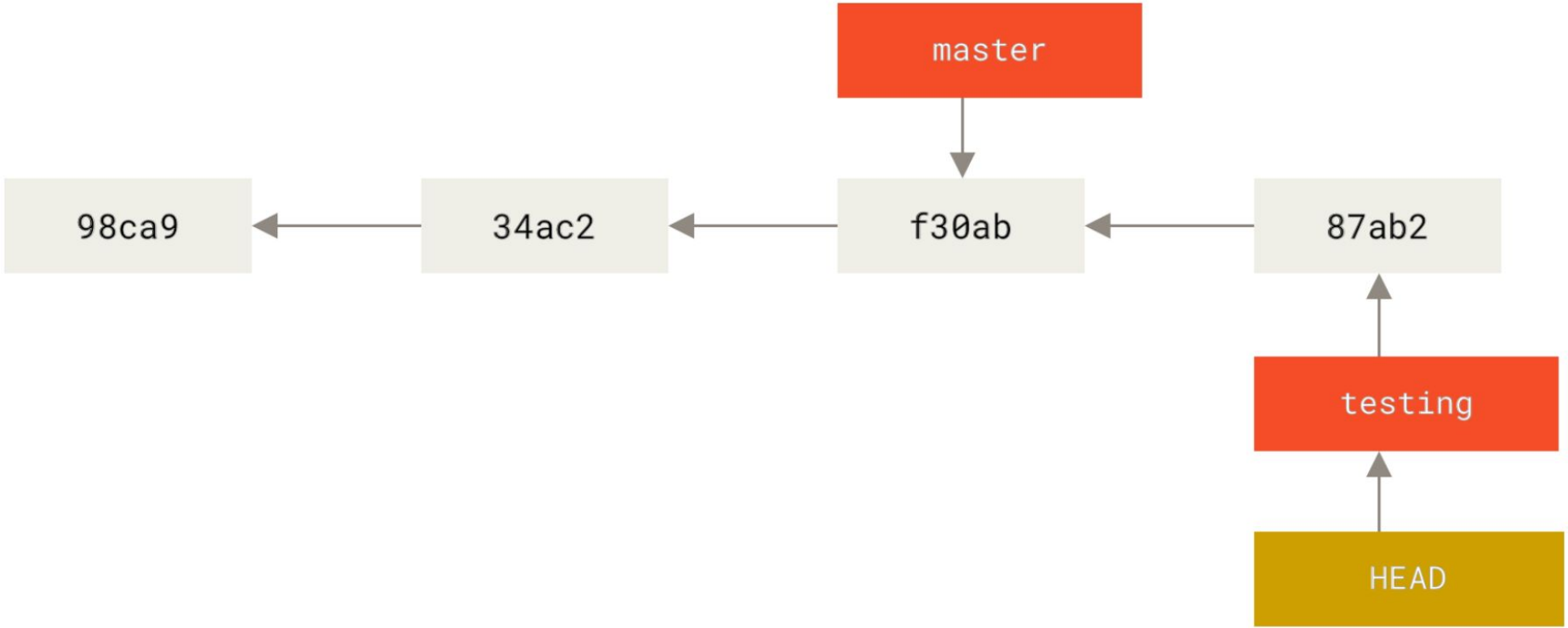


Check which branch you are on?

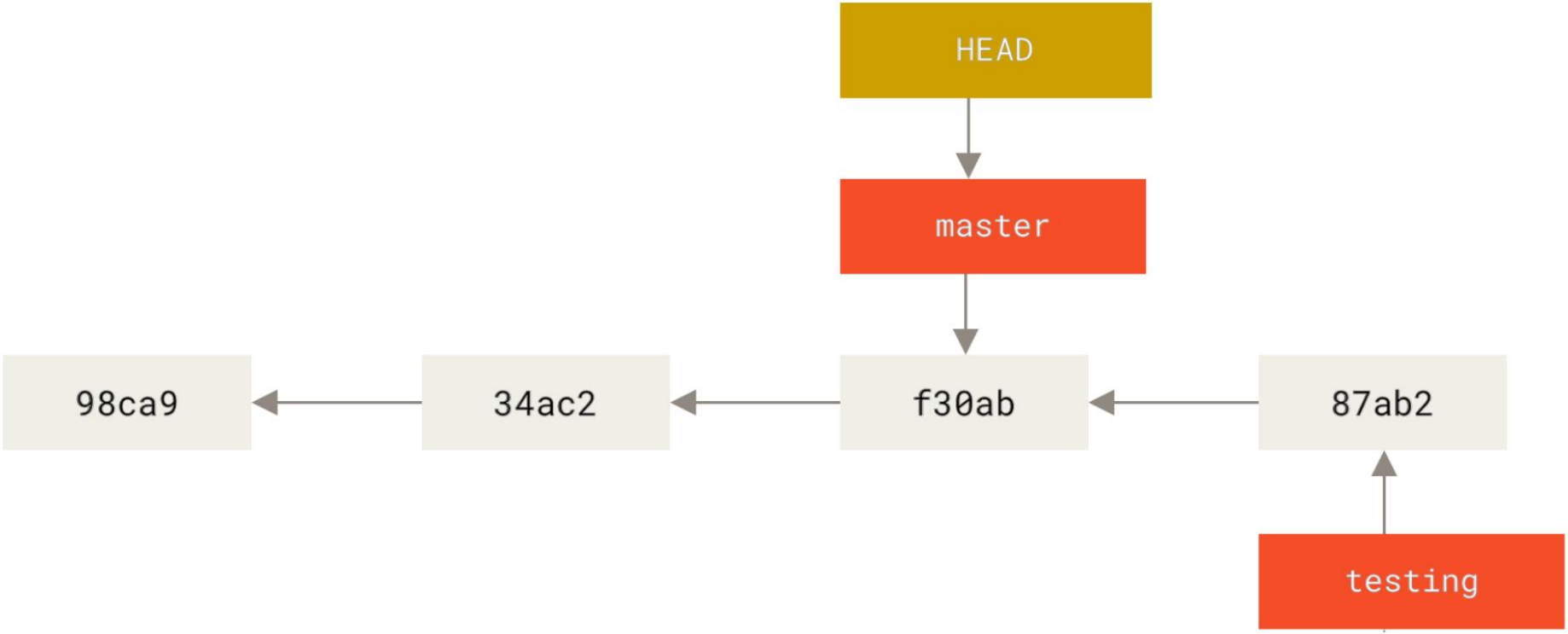



master
98ca9
34ac2
f30ab
testing
HEAD

What is the significance of that? Well, let's do another commit:



This is interesting, because now your testing branch has moved forward, but your master branch still points to the commit you were on when you ran git checkout to switch branches.

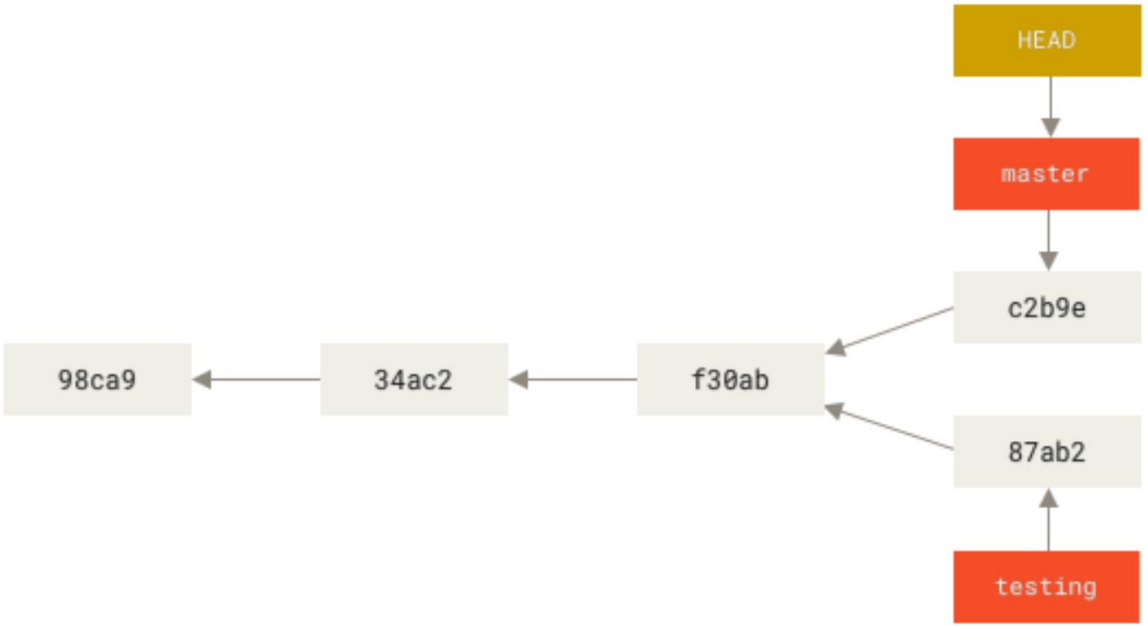


That command did two things:

1. It moved the HEAD pointer back to point to the master branch, and it reverted the files in your working directory back to the snapshot that master points to.

2. This also means the changes you make from this point forward will diverge from an older version of the project. It essentially rewinds the work you've done in your testing branch so you can go in a different direction.

Divergent history



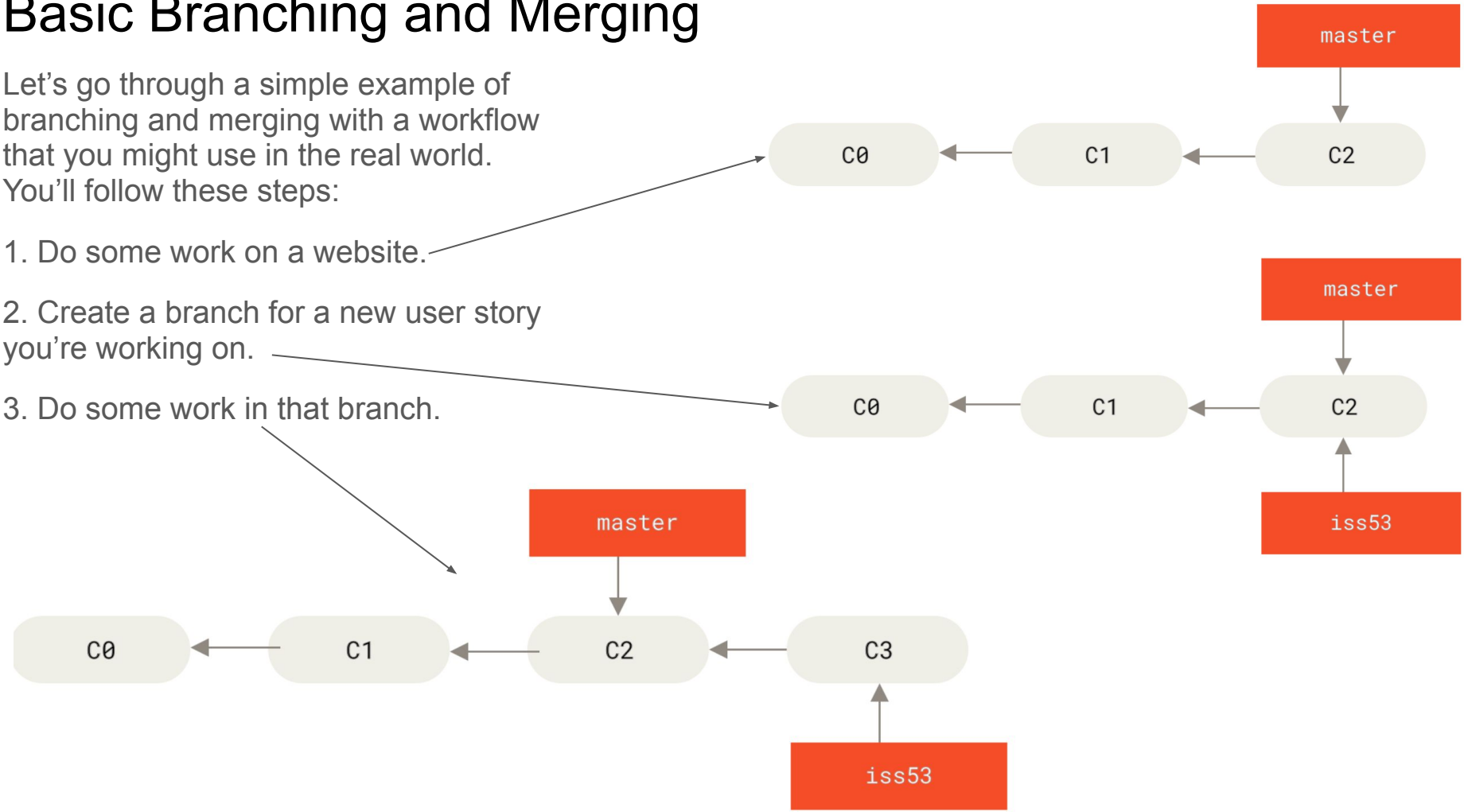
Basic Branching and Merging

Let’s go through a simple example of branching and merging with a workflow that you might use in the real world. You’ll follow these steps:

1. Do some work on a website.

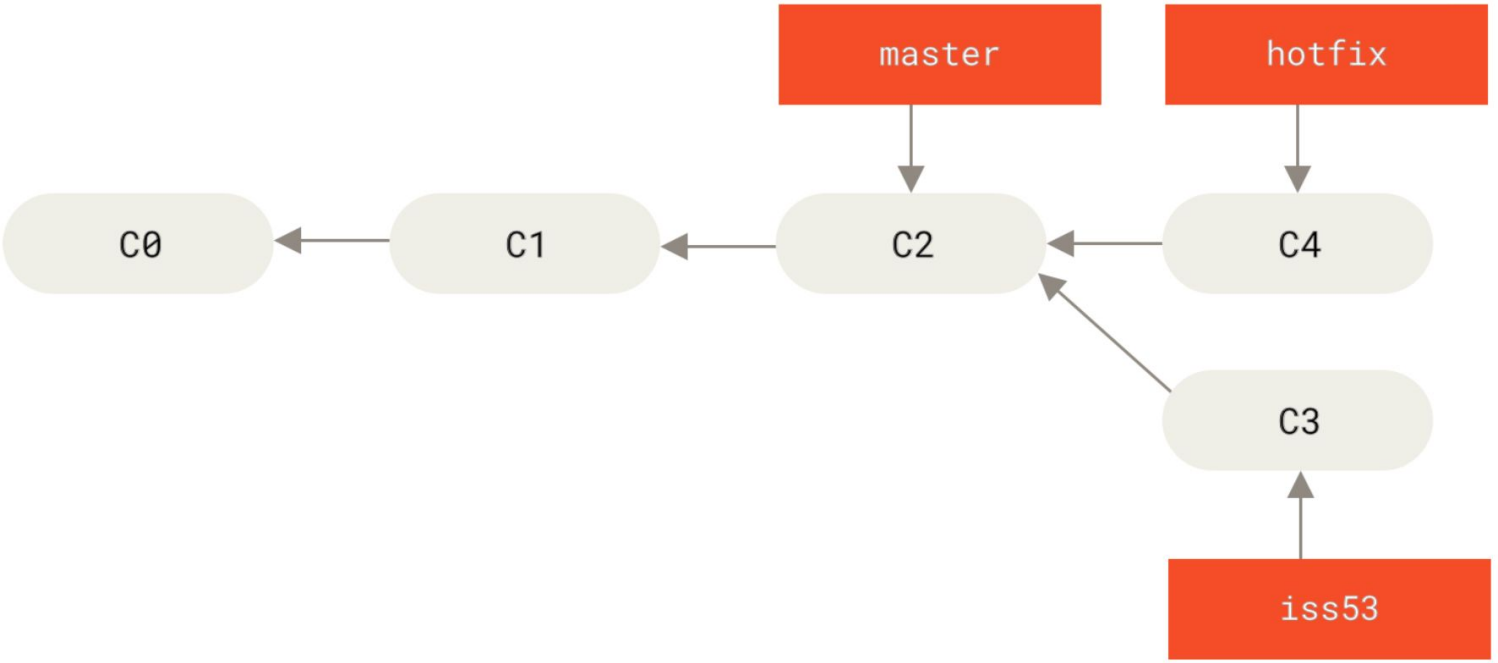
2. Create a branch for a new user story you’re working on.

3. Do some work in that branch.



At this stage, you'll receive a call that another issue is critical and you need a hotfix. You'll do the following:

1. Switch to your production branch.
2. Create a branch to add the hotfix.
3. After it's tested, merge the hotfix branch, and push to production.
4. Switch back to your original user story and continue working.



Merge concept

```
$ git checkout master
$ git merge hotfix
Updating f42c576..3a0874c
Fast-forward
 index.html | 2 ++
 1 file changed, 2 insertions(+)
```

→ You'll notice the phrase "fast-forward" in that merge.

→ Because the commit C4 pointed to by the branch hotfix you merged in was directly ahead of the commit C2 you're on, Git simply moves the pointer forward.

→ To phrase that another way, when you try to merge one commit with a commit that can be reached by following the first commit's history, Git simplifies things by moving the pointer forward because there is no divergent work to merge together — this is called a "fast- forward."

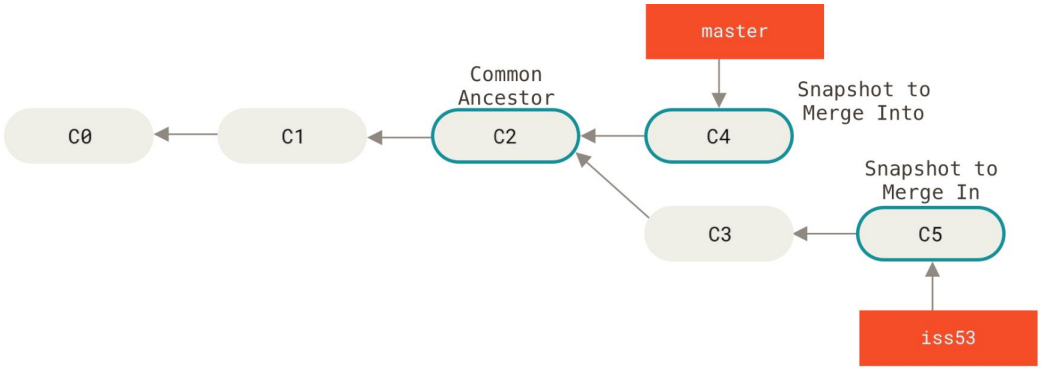
Basic Merging

→ Suppose you've decided that your issue #53 work is complete and ready to be merged into your master branch.

→ In order to do that, you'll merge your iss53 branch into master, much like you merged your hotfix branch earlier.

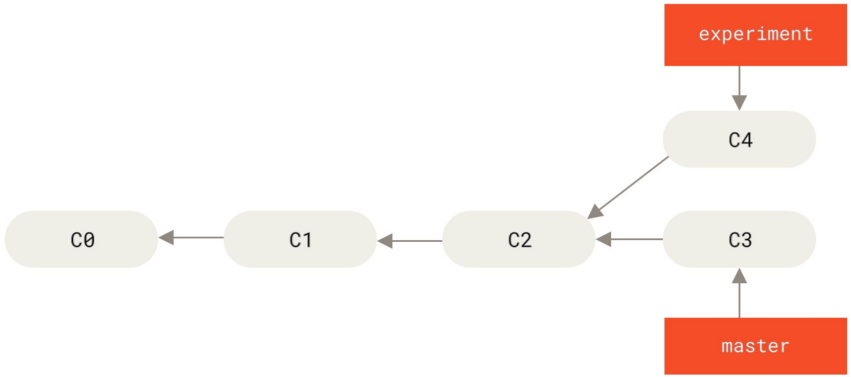
→ All you have to do is check out the branch you wish to merge into and th run the git merge command

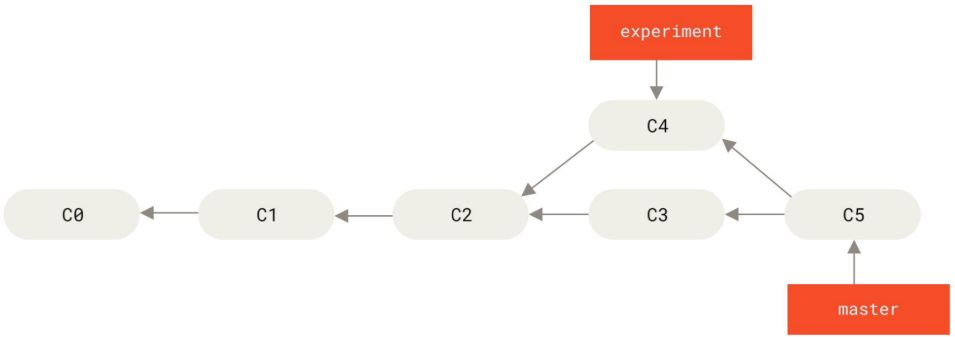
```
$ git checkout master

 Switched to branch 'master'

$ git merge iss53
 Merge made by the 'recursive' strategy.
 index.html |    1 +
 1 file changed, 1 insertion(+)
```



Rebase concept

- In Git, there are two main ways to integrate changes from one branch into another: the merge and the rebase.
- If you go back to previous recording, we can see that there is diverged work and made commits on two different branches.
- However, there is another way: you can take the patch of the change that was introduced in C4 and reapply it on top of C3. This is called rebasing.





Rebasing use case

With the rebase command, you can take all the changes that were committed on one branch and replay them on a different branch.

```
$ git checkout experiment

$ git rebase master

First, rewinding head to replay your work on top of it...

Applying: added staged command
```

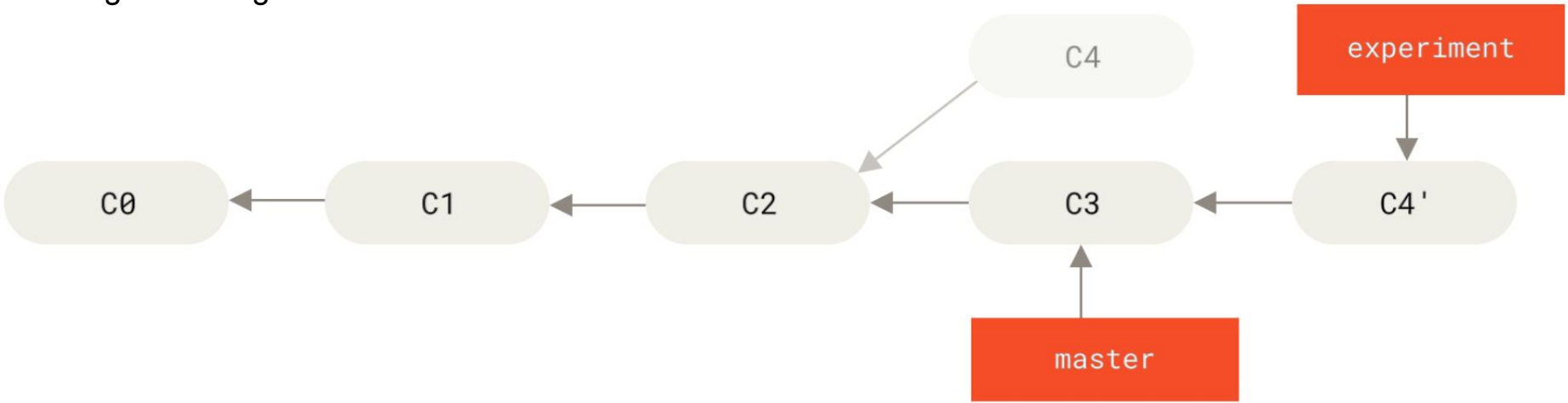
→ This operation works by going to the common ancestor of the two branches (the one you're on and the one you're rebasing onto),

→ Then getting the diff introduced by each commit of the branch you're on,

→ Saving those diffs to temporary files,

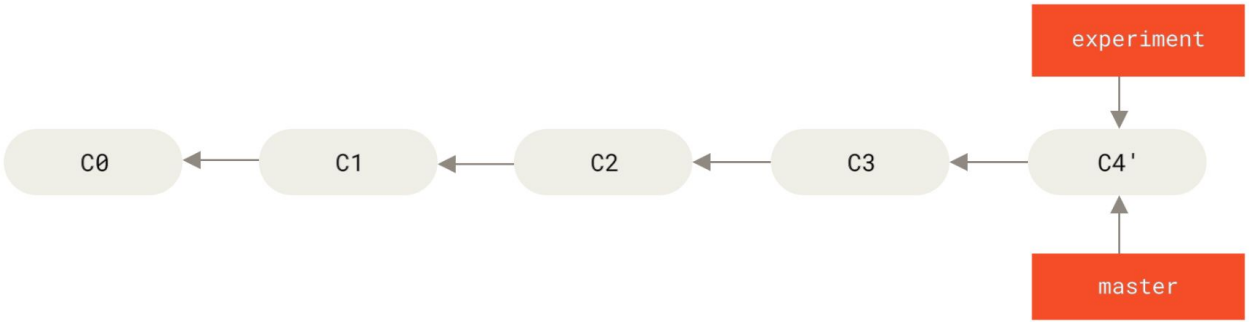
→ Resetting the current branch to the same commit as the branch you are rebasing onto, and finally applying each change in turn.

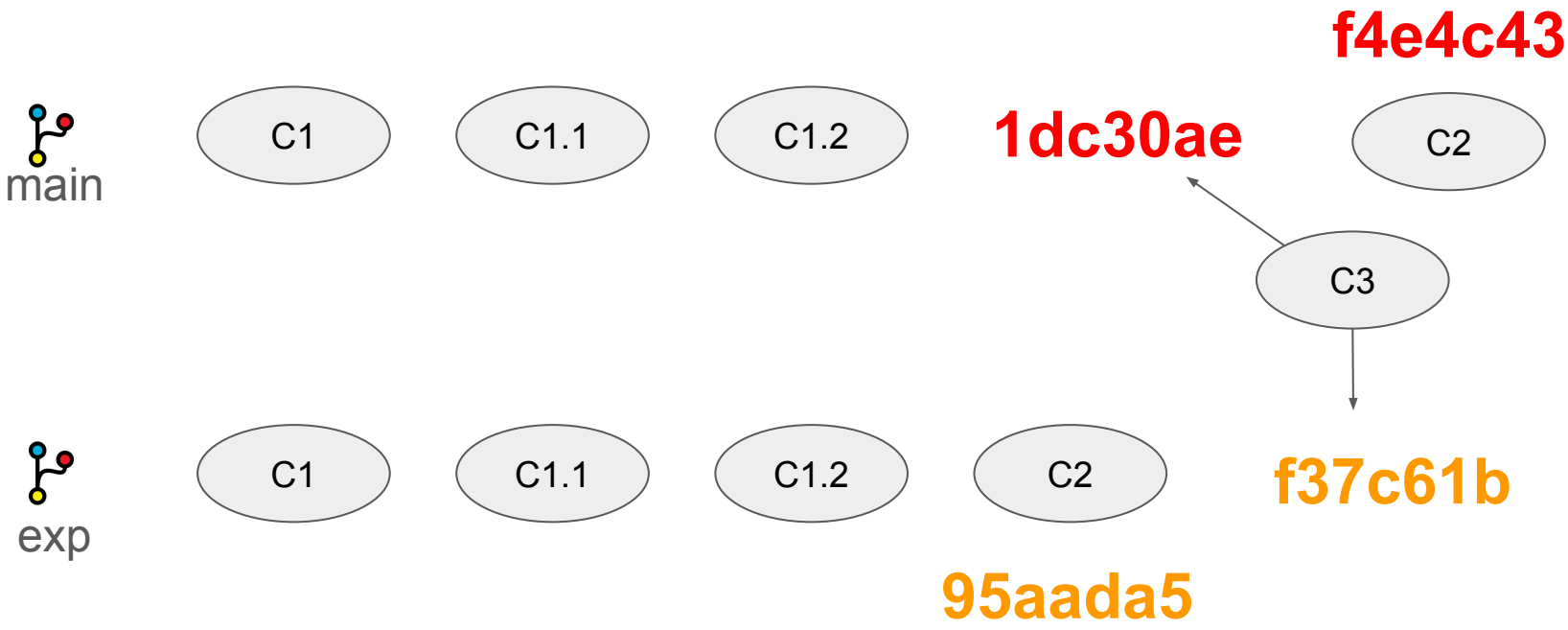
Rebasing the change introduced in C4 onto C3

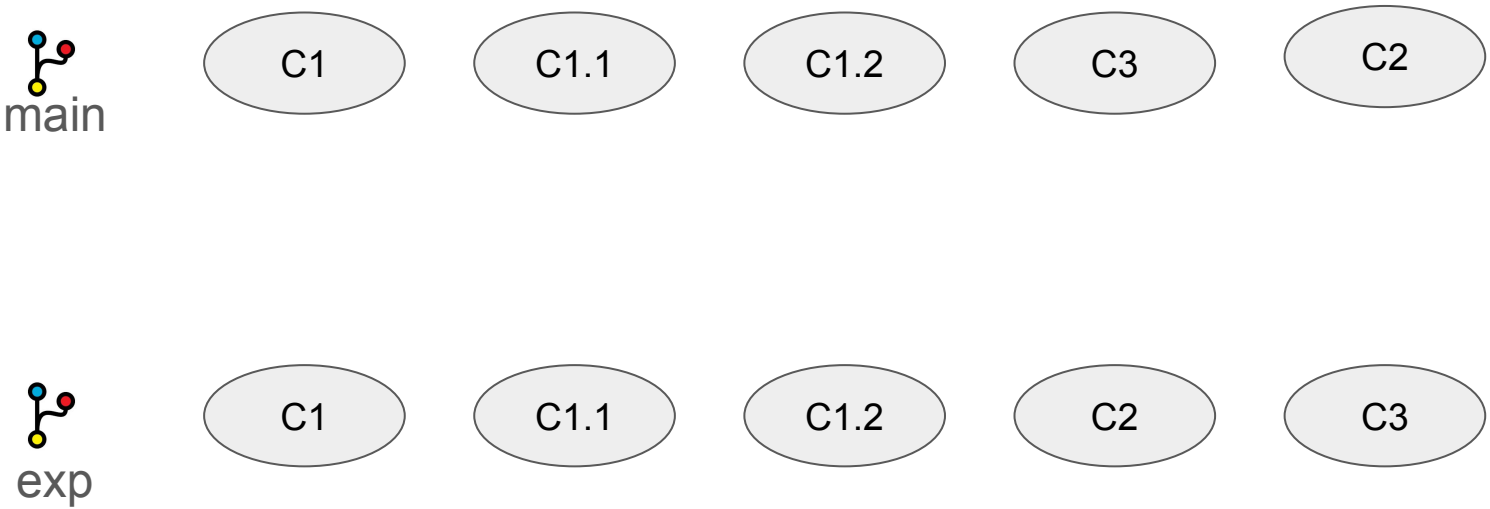


→ At this point, you can go back to the master branch and do a fast-forward merge.

→ Now, the snapshot pointed to by C4' is exactly the same as the one that will be pointed to by C5 in the merge.




f4e4c43
main
C1
C1.1
C1.2
1dc30ae
C2
C3
exp
C1
C1.1
C1.2
C2
f37c61b
95aada5


main
C1
C1.1
C1.2
C3
C2
exp
C1
C1.1
C1.2
C2
C3

GIT Stash

Basic Usage

$ git stash

This command will stash both tracked and untracked changes.

List Stashes:

$ git stash list

Apply Stash:

$ git stash apply

This command will re-apply the most recent stash. It does not remove the stash from the stash list.

To apply a specific stash:

$ git stash apply stash@{index}

Pop Stash:

git stash pop

This command will re-apply the most recent stash and remove it from the stash list.
$ git stash pop stash@{index}

Drop Stash:

$ git stash drop

This command will remove the most recent stash without applying it.

To drop a specific stash:

$ git stash drop stash@{index}

Clear All Stashes:

$ git stash clear

This command will remove all stashes.